**DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN**

**Ingeniería de Software**

**Análisis y Diseño de Software - NRC 30724**

**Guía 2:** Casos de uso y clases

**Proyecto:**

Sistema de Gestión y Control de Transporte de Carga Pesada

**Grupo**


Saraguro Leidy

Ñacato Alexander

Jaguaco Jonathan

Obando Erick


**Instructor:** Ing. Jenny Alexandra Ruiz Robalino

**Fecha:** 05/05/2026

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

## 1. Objetivo

Construir los diagramas de clases de análisis y de secuencia para el Sistema de Gestión y Control de Transporte de Carga Pesada en PlantUML, partiendo del diagrama de casos de uso previamente modelado, con el fin de mantener la trazabilidad entre los modelos funcional, estructural y dinámico, y verificar que la colaboración entre objetos (frontera, control, entidad y repositorio) responda a los requisitos funcionales definidos.

## 2. Desarrollo

### 2.1. Diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "PantallaLogin" <<boundary>> {
  - txtUsuario: String
  - txtContrasena: String
  + clickIniciarSesion()
  + mostrarMensaje(mensaje: String)
  + mostrarMenuRol(rol: String)
}

class "PantallaTransportistas" <<boundary>> {
  - txtId: int
  - txtNombre: String
  - txtDocumentacion: String
  + clickCrear()
  + clickEditar()
  + clickEliminar()
  + clickConsultar()
  + mostrarMensaje(mensaje: String)
  + mostrarTabla(transportistas: List<Transportista>)
}

class "PantallaViajes" <<boundary>> {
  - txtOrigen: String
  - txtDestino: String
  - txtFecha: Date
  - txtCarga: String
  + clickCrear()
  + clickCancelar()
  + clickReprogramar()
  + clickConsultar()
  + clickAsignarTransportista()
  + clickPlanificarRuta()
  + mostrarMensaje(mensaje: String)
  + mostrarTabla(viajes: List<Viaje>)
```

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

```
}

class "PantallaMonitoreo" <<boundary>> {
  - mapaMostrado: boolean
  + clickMonitorear()
  + clickModificarRuta()
  + mostrarUbicacion(ubicacion: String)
  + mostrarAlerta(alerta: String)
}

class "ControlSesion" <<control>> {
  + iniciarSesion(usuario: String, contrasena: String): Resultado
  + validarCredenciales(usuario: String, contrasena: String): boolean
  + obtenerRol(usuario: String): String
}

class "ControlTransportistas" <<control>> {
  + crearTransportista(id: int, nombre: String, doc: String): Resultado
  + editarTransportista(id: int, nombre: String, doc: String): Resultado
  + eliminarTransportista(id: int): Resultado
  + consultarTransportistas(): List<Transportista>
  + validarDocumentacion(doc: String): boolean
}

class "ControlViajes" <<control>> {
  + crearViaje(origen: String, destino: String, fecha: Date, carga: String): Resultado
  + cancelarViaje(id: int): Resultado
  + reprogramarViaje(id: int, fecha: Date): Resultado
  + consultarViajes(): List<Viaje>
  + asignarTransportista(idViaje: int, idTransportista: int): Resultado
  + planificarRuta(idViaje: int, ruta: String): Resultado
}

class "ControlMonitoreo" <<control>> {
  + monitorearViaje(idViaje: int): Resultado
  + modificarRuta(idViaje: int, nuevaRuta: String): Resultado
  + generarAlerta(idViaje: int, tipo: String): void
  + calcularETA(idViaje: int): String
}

class "Usuario" <<entity>> {
  - id: int
  - nombre: String
  - contrasena: String
  - rol: String
  + Usuario(id: int, nombre: String, contrasena: String, rol: String)
  + getId(): int
  + getNombre(): String
  + getRol(): String
}

class "Transportista" <<entity>> {
  - id: int
  - nombre: String
```

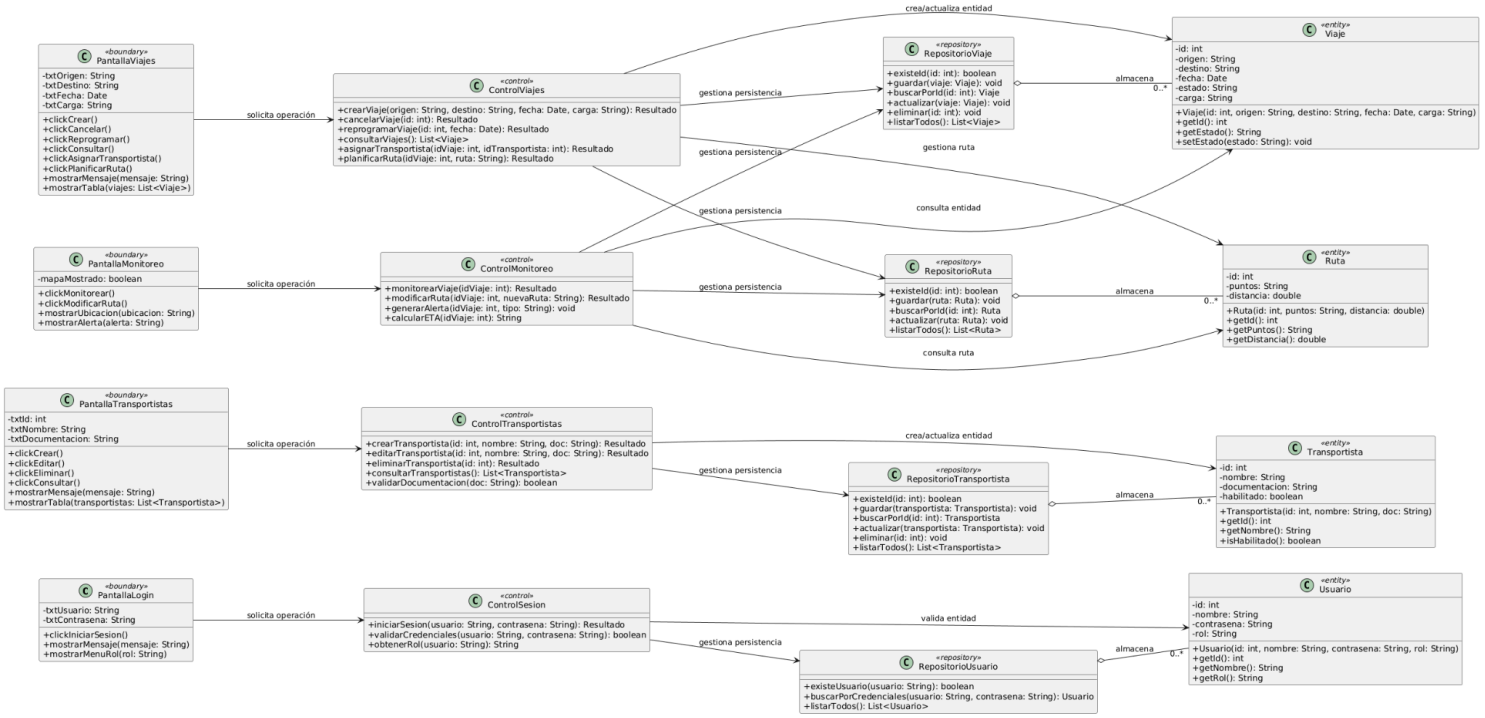
| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

```
  - documentacion: String
  - habilitado: boolean
  + Transportista(id: int, nombre: String, doc: String)
  + getId(): int
  + getNombre(): String
  + isHabilitado(): boolean
}

class "Viaje" <<entity>> {
  - id: int
  - origen: String
  - destino: String
  - fecha: Date
  - estado: String
  - carga: String
  + Viaje(id: int, origen: String, destino: String, fecha: Date, carga: String)
  + getId(): int
  + getEstado(): String
  + setEstado(estado: String): void
}

class "Ruta" <<entity>> {
  - id: int
  - puntos: String
  - distancia: double
  + Ruta(id: int, puntos: String, distancia: double)
  + getId(): int
  + getPuntos(): String
  + getDistancia(): double
}

class "RepositorioUsuario" <<repository>> {
  + existeUsuario(usuario: String): boolean
  + buscarPorCredenciales(usuario: String, contrasena: String): Usuario
  + listarTodos(): List<Usuario>
}

class "RepositorioTransportista" <<repository>> {
  + existeId(id: int): boolean
  + guardar(transportista: Transportista): void
  + buscarPorId(id: int): Transportista
  + actualizar(transportista: Transportista): void
  + eliminar(id: int): void
  + listarTodos(): List<Transportista>
}

class "RepositorioViaje" <<repository>> {
  + existeId(id: int): boolean
  + guardar(viaje: Viaje): void
  + buscarPorId(id: int): Viaje
  + actualizar(viaje: Viaje): void
  + eliminar(id: int): void
  + listarTodos(): List<Viaje>
}
```

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

```
class "RepositorioRuta" <<repository>> {
  + existeId(id: int): boolean
  + guardar(ruta: Ruta): void
  + buscarPorId(id: int): Ruta
  + actualizar(ruta: Ruta): void
  + listarTodos(): List<Ruta>
}

' Boundaries a Controls
PantallaLogin --> ControlSesion : solicita operación
PantallaTransportistas --> ControlTransportistas : solicita operación
PantallaViajes --> ControlViajes : solicita operación
PantallaMonitoreo --> ControlMonitoreo : solicita operación

' Controls a Entities
ControlSesion --> Usuario : valida entidad
ControlTransportistas --> Transportista : crea/actualiza entidad
ControlViajes --> Viaje : crea/actualiza entidad
ControlViajes --> Ruta : gestiona ruta
ControlMonitoreo --> Viaje : consulta entidad
ControlMonitoreo --> Ruta : consulta ruta

' Controls a Repositorios
ControlSesion --> RepositorioUsuario : gestiona persistencia
ControlTransportistas --> RepositorioTransportista : gestiona persistencia
ControlViajes --> RepositorioViaje : gestiona persistencia
ControlViajes --> RepositorioRuta : gestiona persistencia
ControlMonitoreo --> RepositorioViaje : gestiona persistencia
ControlMonitoreo --> RepositorioRuta : gestiona persistencia

' Repositorios a Entities
RepositorioUsuario o-- "0..*" Usuario : almacena
RepositorioTransportista o-- "0..*" Transportista : almacena
RepositorioViaje o-- "0..*" Viaje : almacena
RepositorioRuta o-- "0..*" Ruta : almacena

@enduml
```

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |




**Explicación Breve:**

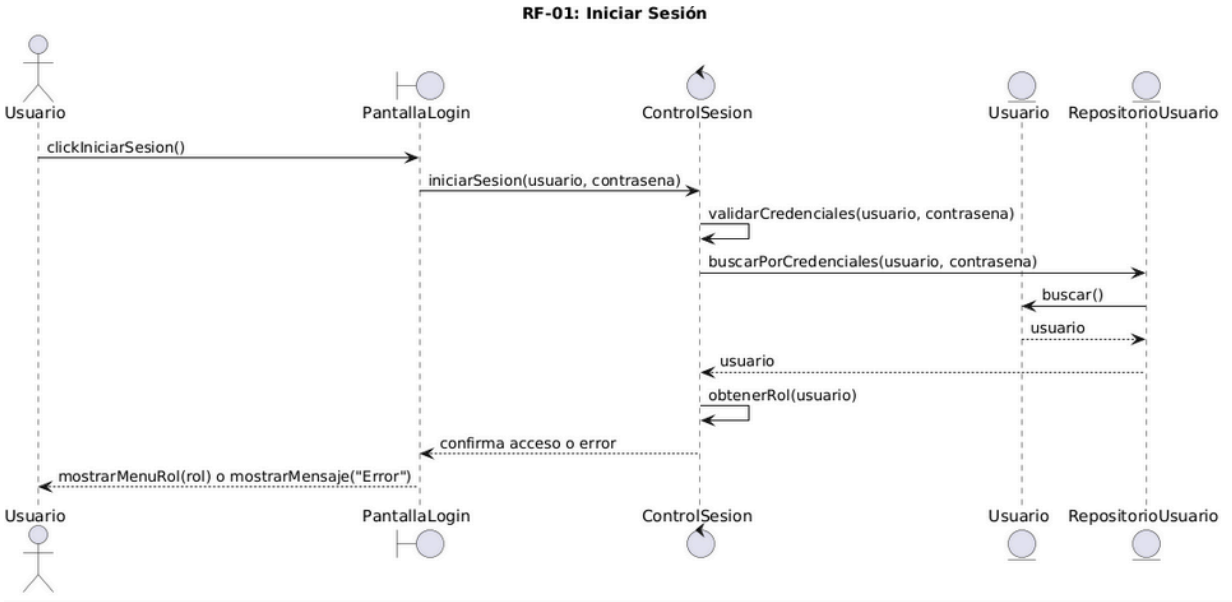
- Las clases PantallaLogin, PantallaTransportistas, PantallaViajes y PantallaMonitoreo (<<boundary>>) representan las interfaces visuales de cada módulo con sus campos de entrada y métodos asociados a cada botón de interacción del usuario.
- Las clases ControlSesion, ControlTransportistas, ControlViajes y ControlMonitoreo (<<control>>) coordinan la lógica de cada caso de uso gestionando validaciones, operaciones y generación de alertas.
- Las clases Usuario, Transportista, Viaje y Ruta (<<entity>>) modelan la información del dominio del negocio con sus atributos privados, constructores y métodos de acceso.
- Las clases RepositorioUsuario, RepositorioTransportista, RepositorioViaje y RepositorioRuta (<<repository>>) gestionan la persistencia ofreciendo operaciones de guardar, buscar, actualizar, eliminar y listar, almacenando cada una una colección de cero o más entidades.
- Las relaciones muestran que cada Boundary solicita operaciones a su Control, el Control crea o actualiza la entidad y delega la persistencia al Repositorio, y cada Repositorio almacena una colección de cero o más entidades del sistema.

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

## 2.2 Diagrama de Secuencias

### 2.2.1. Secuencia: Iniciar Sesión

```
@startuml
title RF-01: Iniciar Sesión

actor Usuario as U
boundary "PantallaLogin" as B
control "ControlSesion" as CT
entity "Usuario" as E
entity "RepositorioUsuario" as R

U -> B : clickIniciarSesion()
B -> CT : iniciarSesion(usuario, contrasena)
CT -> CT : validarCredenciales(usuario, contrasena)
CT -> R : buscarPorCredenciales(usuario, contrasena)
R -> E : buscar()
E --> R : usuario
R --> CT : usuario
CT -> CT : obtenerRol(usuario)
CT --> B : confirma acceso o error
B --> U : mostrarMenuRol(rol) o mostrarMensaje("Error")
@enduml
```



| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

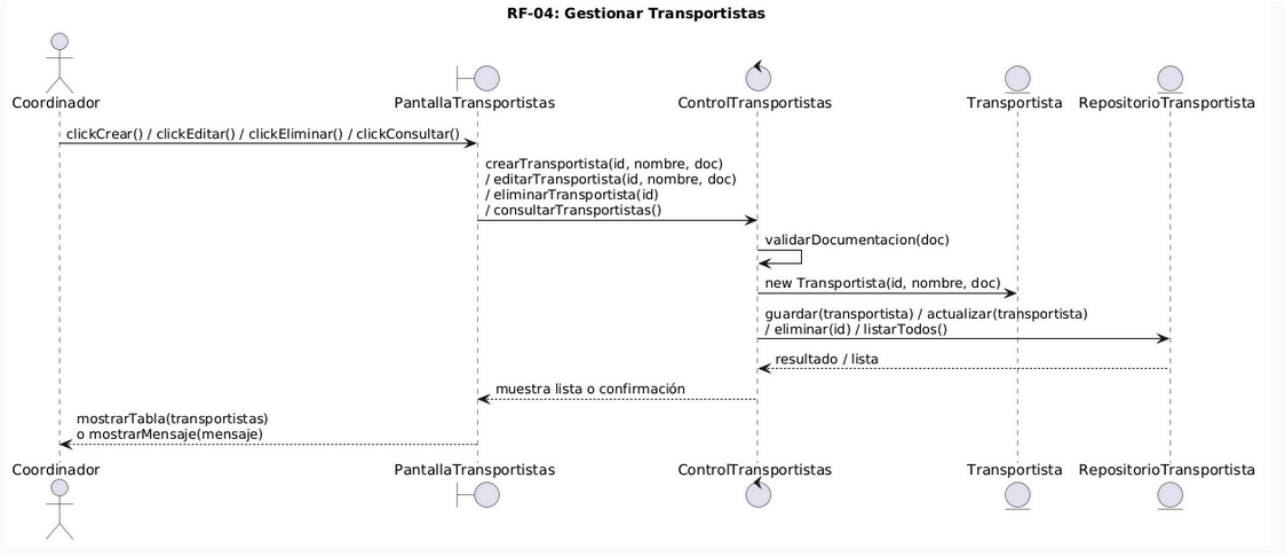
**Explicación Breve:**

- El Usuario activa clickIniciarSesion() en la PantallaLogin (boundary).
- La pantalla delega al ControlSesion llamando a iniciarSesion().
- El control valida las credenciales internamente y consulta el RepositorioUsuario.
- El repositorio busca el Usuario en la entidad y retorna el resultado al control.
- El control obtiene el rol y retorna confirmación o error a la pantalla.
- La pantalla muestra el menú según el rol o el mensaje de error al Usuario.

### 2.2.2 Secuencia: Gestionar Transportistas

```
@startuml
title RF-04: Gestionar Transportistas

actor Coordinador as C
boundary "PantallaTransportistas" as B
control "ControlTransportistas" as CT
entity "Transportista" as E
entity "RepositorioTransportista" as R

C -> B : clickCrear() / clickEditar() / clickEliminar() / clickConsultar()
B -> CT : crearTransportista(id, nombre, doc) \n/ editarTransportista(id,
nombre, doc) \n/ eliminarTransportista(id) \n/ consultarTransportistas()
CT -> CT : validarDocumentacion(doc)
CT -> E : new Transportista(id, nombre, doc)
CT -> R : guardar(transportista) / actualizar(transportista) \n/
eliminar(id) / listarTodos()
R --> CT : resultado / lista
CT --> B : muestra lista o confirmación
B --> C : mostrarTabla(transportistas) \no mostrarMensaje(mensaje)
@enduml
```

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |




**Explicación Breve:**

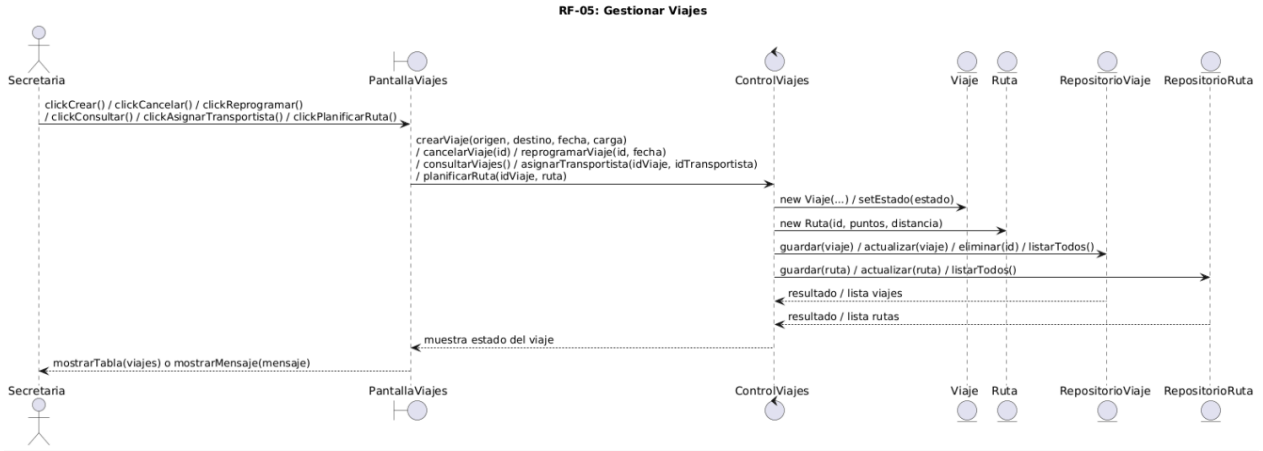
- El Coordinador selecciona la operación deseada en la PantallaTransportistas.
- La pantalla envía los datos al ControlTransportistas con la operación correspondiente.
- El control valida la documentación y crea o manipula la entidad Transportista.
- Delega la persistencia al RepositorioTransportista (guardar, actualizar, eliminar o listar).
- El repositorio retorna el resultado o la lista al control.
- El control retorna confirmación o listado a la pantalla para mostrarlo al actor.

### 2.2.3 Secuencia: Gestionar Viajes

```
@startuml
title RF-05: Gestionar Viajes

actor Secretaria as S
boundary "PantallaViajes" as B
control "ControlViajes" as CT
entity "Viaje" as EV
entity "Ruta" as ER
entity "RepositorioViaje" as RV
entity "RepositorioRuta" as RR
S -> B : clickCrear() / clickCancelar() / clickReprogramar() \n/ clickConsultar() /
clickAsignarTransportista() / clickPlanificarRuta()
```

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

```
B -> CT : crearViaje(origen, destino, fecha, carga) \n/ cancelarViaje(id) /
reprogramarViaje(id, fecha) \n/ consultarViajes() / asignarTransportista(idViaje,
idTransportista) \n/ planificarRuta(idViaje, ruta)
CT -> EV : new Viaje(...) / setEstado(estado)
CT -> ER : new Ruta(id, puntos, distancia)
CT -> RV : guardar(viaje) / actualizar(viaje) / eliminar(id) / listarTodos()
CT -> RR : guardar(ruta) / actualizar(ruta) / listarTodos()
RV --> CT : resultado / lista viajes
RR --> CT : resultado / lista rutas
CT --> B : muestra estado del viaje
B --> S : mostrarTabla(viajes) o mostrarMensaje(mensaje)
@enduml
```



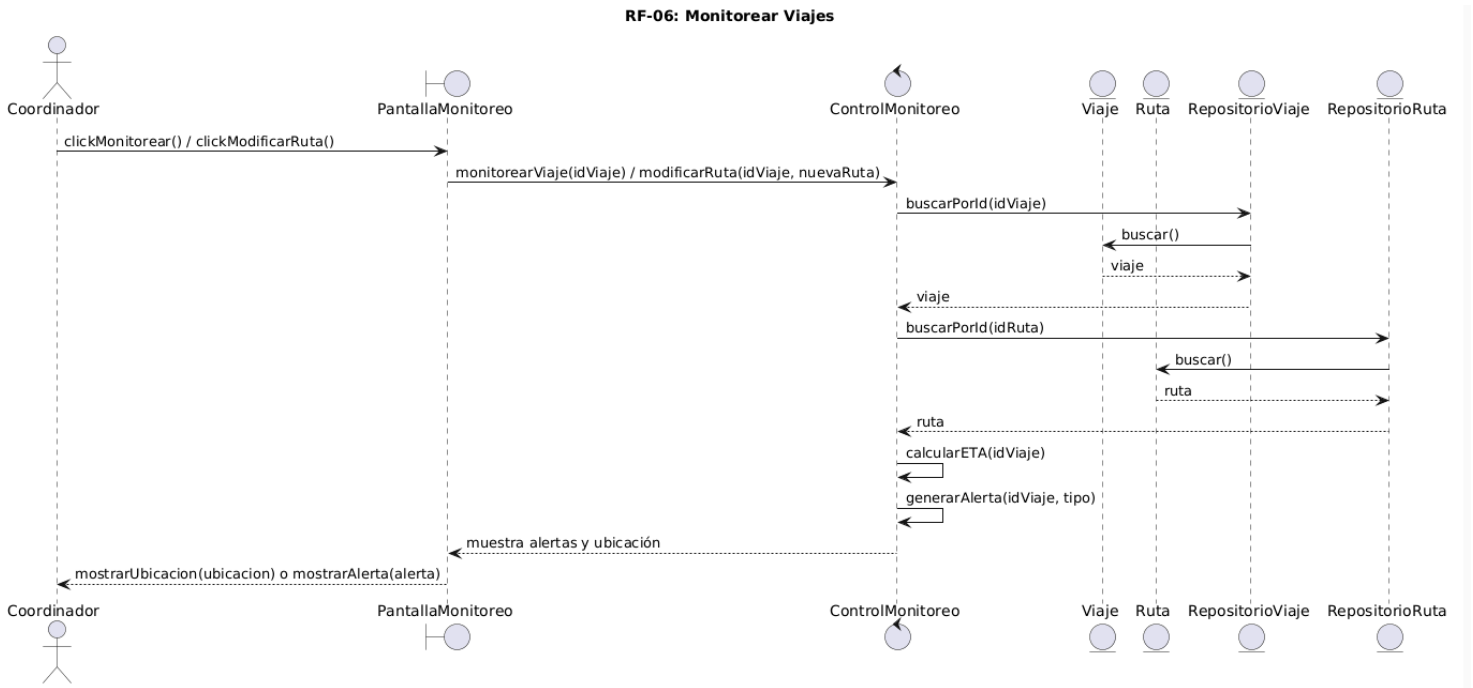

## Explicación Breve:

- La Secretaria selecciona la operación sobre viajes en la PantallaViajes.
- La pantalla delega al ControlViajes con los datos del viaje o ruta.
- El control crea o actualiza las entidades Viaje y Ruta según la operación.
- Delega la persistencia a los repositorios RepositorioViaje y RepositorioRuta.
- Los repositorios retornan los resultados al control.
- El control retorna el estado del viaje o listados a la pantalla para mostrarlo a la Secretaria.

| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

### 2.2.4 Secuencia: Monitorear Viajes

```
@startuml
title RF-06: Monitorear Viajes

actor Coordinador as C
boundary "PantallaMonitoreo" as B
control "ControlMonitoreo" as CT
entity "Viaje" as EV
entity "Ruta" as ER
entity "RepositorioViaje" as RV
entity "RepositorioRuta" as RR
C -> B : clickMonitorear() / clickModificarRuta()
B -> CT : monitorearViaje(idViaje) / modificarRuta(idViaje, nuevaRuta)
CT -> RV : buscarPorId(idViaje)
RV -> EV : buscar()
EV --> RV : viaje
RV --> CT : viaje
CT -> RR : buscarPorId(idRuta)
RR -> ER : buscar()
ER --> RR : ruta
RR --> CT : ruta
CT -> CT : calcularETA(idViaje)
CT -> CT : generarAlerta(idViaje, tipo)
CT --> B : muestra alertas y ubicación
B --> C : mostrarUbicacion(ubicacion) o mostrarAlerta(alerta)
@enduml
```



| | |
|---|---|
| **TALLER 2**<br><br>**UNIDAD 1** | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

**Explicación Breve:**

- El Coordinador activa el monitoreo o modifica la ruta desde la PantallaMonitoreo.
- La pantalla delega al ControlMonitoreo con el identificador del viaje.
- El control consulta el RepositorioViaje y el RepositorioRuta para obtener el estado actual.
- Los repositorios buscan en sus respectivas entidades Viaje y Ruta y retornan los datos.
- El control calcula el tiempo estimado de llegada y genera alertas si detecta incidencias.
- Retorna la ubicación y alertas a la pantalla para mostrarlas al Coordinador.

### 2.3 Trazabilidad

| RF | Caso de uso | Actor | Prioridad | Criterio de aceptación | Evidencia PlantUML |
|---|---|---|---|---|---|
| RF-01 | Iniciar Sesión | Usuario | Alta | El sistema valida credenciales y otorga acceso según el rol asignado; credenciales incorrectas muestran mensaje de error. | UC1 |
| RF-04 | Gestionar Transportistas | Coordinador / Secretaria / Transportista | Alta | El sistema registra, edita o elimina un transportista con documentación válida y notifica inconsistencias. | UC2 |
| RF-05 | Gestionar Viajes | Secretaria / Gerente / Transportista / Cliente | Alta | El sistema crea, consulta, reprograma o cancela un viaje con ruta y transportista asignado correctamente. | UC3 |
| RF-06 | Monitorear Viajes | Coordinador / Transportista | Media | El sistema muestra la ubicación GPS en tiempo real y genera alertas ante retrasos o desvíos detectados. | UC4 |

| | |
|---|---|
| TALLER 2<br><br>UNIDAD 1 | **Departamento:** Ciencias de la computación |
| | **Carrera:** Ingeniería de software<br>**Asignatura:** Análisis y Diseño de software<br>**NRC:** 30724 |
| | **Apellidos y nombres del estudiante:**<br>Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick |

## 3. Conclusión técnica

- La implementación del sistema TransControl permitió conectar el diagrama de casos de uso con el diagrama de clases de análisis y con los diagramas de secuencia, donde cada caso de uso (Iniciar Sesión, Gestionar Transportistas, Gestionar Viajes y Monitorear Viajes) se tradujo en responsabilidades asignadas a las clases PantallaLogin, PantallaTransportistas, PantallaViajes y PantallaMonitoreo (frontera), sus controles correspondientes (control), las entidades Usuario, Transportista, Viaje y Ruta (entidad) y sus respectivos repositorios (repositorio).
- Cada operación modelada en los diagramas de secuencia fue verificada contra los métodos existentes en el diagrama de clases, garantizando que cada mensaje enviado entre objetos tuviera su correspondencia exacta en un método definido, lo que obligó a mantener coherencia en nombres, tipos de parámetros y valores de retorno desde el inicio del modelado.
- Esta trazabilidad asegura que los diagramas no son piezas aisladas sino un modelo coherente que puede servir como base para implementar un sistema funcional de gestión de transporte de carga pesada en cualquier entorno de desarrollo, cumpliendo con los requisitos funcionales definidos en el SRS del proyecto TransControl.

## 4. Recomendaciones

- Para próximos talleres se recomienda unificar los tipos de datos de los atributos desde el inicio del modelado, en especial los identificadores y fechas, decidiendo si serán entero, cadena o tipo Date, y mantener esa decisión consistente en todas las clases (entidad, repositorio, control) y en los parámetros de los mensajes de secuencia, evitando mezclas que generen inconsistencias difíciles de rastrear entre diagramas.
- Se sugiere compilar y validar el diagrama de clases de análisis antes de elaborar los diagramas de secuencia, revisando que los nombres de los métodos, los tipos de parámetros y los valores de retorno estén correctamente definidos, porque cualquier error en la clase se arrastra automáticamente a todas las secuencias que la referencian y puede comprometer la trazabilidad del modelo completo.
- También es recomendable documentar en una tabla aparte la correspondencia entre cada mensaje de secuencia y el método de la clase destino, funcionando como lista de verificación adicional para garantizar que ningún mensaje quede sin respaldo en el diagrama de clases, evitando así rehacer diagramas al final del proceso de modelado.